

CLINICAL DISCHARGE SUMMARY AGENT — DESIGN NOTES

WHAT IS BUILT

An agentic AI system that reads raw hospital records (PDFs containing typed discharge summaries, handwritten ER charts, lab reports, and nursing notes) and produces a structured, clinically safe discharge summary draft for clinician review. Built from scratch in Python with no LangChain or CrewAI.

AGENT LOOP DESIGN

The system uses a hybrid agent architecture — a deliberate choice made after observing that a pure LLM agent loop on noisy OCR text produces hallucinations regardless of prompt strength.

Layer 1 — Deterministic Python Extraction

All mechanical extraction is done in Python directly. Section parsing uses keyword-based boundary detection to locate clinical sections such as DIAGNOSIS, HISTORY, COURSE IN THE HOSPITAL, CONDITION AT DISCHARGE, and FOLLOW-UP INSTRUCTIONS. Table extraction is handled via pdfplumber for structured content like medication tables. Pages where the PDF contains no embedded text (handwritten forms, scanned charts) are automatically identified and passed through Tesseract OCR as a fallback. This layer is fast, reproducible, and consumes zero LLM tokens. It handles the parts of the problem that do not require intelligence.

Layer 2 — Medical API Enrichment (RAG)

Rather than asking the LLM to recall medical knowledge from training data, the system retrieves grounded facts from three free APIs. The NLM ICD-10 API validates and assigns standard codes to every diagnosis. The RxNorm API resolves Indian brand drug names to their generic equivalents and drug class. OpenFDA checks for documented interactions between discharge medications. This is the retrieval-augmented generation layer. The agent does not guess what an unfamiliar brand name means — it looks it up. This approach eliminates the most common hallucination pattern observed during development, where the model would substitute a plausible-sounding drug name for an unfamiliar brand name.

Layer 3 — LLM Agent Loop (Groq / Llama 3.3 70B)

Used for conflict detection, flag enrichment, and cases requiring language understanding. Implements a real tool-calling loop where the LLM decides which tools to call based on what it finds. The loop follows this sequence: load the document, extract structured data from known pages, validate each diagnosis against ICD-10, normalise each medication name via RxNorm, check key drug pairs for interactions via OpenFDA, flag any conflicts found between document sections, flag any required fields not found, then call `finalize_summary` to produce the structured output. A step cap of 25 iterations is enforced. Every step is logged to a JSON

trace file with timestamp, thought, action, arguments, and result, providing full observability into why the agent made each decision.

NO-FABRICATION GUARDRAIL

This is the most important design constraint and is enforced at multiple levels simultaneously.

Prompt-level enforcement. The system prompt contains explicit rules that the LLM must follow. It is instructed never to invent, assume, or infer clinical facts. If a value is not explicitly stated in the source document, it must be flagged as missing. Garbled OCR text means the field is missing — not an invitation to guess. Medication names must be copied verbatim and never substituted with a drug name from medical training knowledge.

Output schema enforcement. The `finalize_summary` tool rejects calls where key fields contain obvious placeholder strings such as "History" or "Hospital course", forcing the agent to use real extracted values or explicit missing flags before the summary can be saved.

Explicit missing field protocol. Every required field that cannot be found in any source page is populated with the exact phrase: [MISSING — REQUIRES CLINICIAN REVIEW]. This is never left blank. A blank field could be mistaken for a field that was checked and found empty. An explicit flag cannot be misread.

Source attribution on conflicts. The system tracks which page each value came from. When the same clinical field appears with different values in different parts of the document, both values are flagged with their sources. For example: CONFLICT [Sodium level]: '127 mmol/L' from Page 1 discharge summary vs '114 mmol/L' from Page 28 ABG — requires clinician review. The agent never resolves a conflict by picking one value. Both are surfaced for the clinician to adjudicate.

API-grounded validation. Drug names and diagnoses are validated against external authoritative databases rather than the LLM's training data. This prevents hallucination at the most clinically dangerous point — medication reconciliation.

FAILURE AND CONFLICT HANDLING

OCR failures. All 71 pages of the input PDF required OCR as the document contains no embedded text. Pages where OCR returns fewer than 50 characters are flagged with an extraction warning noting that content may be incomplete. The agent does not attempt to interpret noise as clinical data.

API timeouts. Every external API call is wrapped with a 10-second timeout and error handling. On failure the agent receives a descriptive error string and continues processing — it flags the relevant field for manual review rather than crashing the run.

Token rate limiting. The Groq free tier enforces a 100,000 token per day limit. The loop handles this by truncating tool result messages before storing them in conversation history, trimming the message context window to the last 12 messages, and on a rate limit error, trimming more aggressively to the last 4 messages before retrying.

Step cap. The agent loop enforces a maximum of 25 steps. If the cap is reached, the summary is saved in its current state with a flag noting it may be incomplete. A partial summary with honest flags is safer than a failed run with no output.

Conflicting clinical data. When the same field appears with different values in different parts of the document, the agent records both values and their source pages via the `flag_conflict` tool, includes both in the `clinician_flags` array of the output, and never arbitrarily selects one value as authoritative.

WHAT IS COMPLETE

PDF ingestion with OCR fallback using PyMuPDF and Tesseract. Python-based clinical section extraction. ICD-10 diagnosis validation and coding via the NLM API. Drug name normalisation resolving Indian brand names to generics via RxNorm and a local brand name database. Drug interaction checking via OpenFDA. No-fabrication enforcement at prompt, schema, and validation layers. Conflict detection with source attribution. Missing field flagging with explicit markers. Critical alert detection for sodium below 120 mmol/L, HbA1c above 10%, discharge against medical advice, and pending results. Step cap and iteration control with graceful degradation. Full JSON trace of every agent step. Structured JSON output with ICD-10 codes and generic drug names. Formatted clinical discharge summary PDF output generated from the JSON. Gradio web interface for PDF upload, summary generation, and download. Five-dimension feedback scoring stored in SQLite. Feedback retrieval infrastructure for few-shot improvement in future runs.

WHAT IS NOT COMPLETE AND WHAT WOULD COME NEXT

Hospital course extraction. The hospital course section spans two pages and the OCR boundary detection does not reliably locate it. The field is flagged as missing rather than fabricated. The fix would be to train a lightweight span extractor on clinical document structure or use a higher-DPI OCR pass on the key pages.

Exact admission and discharge dates. Only month and year are readable from the OCR output. The typed document does not contain date fields in a consistent machine-readable location. The fix would be improved section boundary detection combined with a date-specific regex pass.

Patient demographics. Name, age, and weight are not present in any reliably readable page. These fields are correctly flagged as missing. The fix would be to apply improved handwriting OCR with pre-processing on the admission record pages.

Quantified RL improvement. The feedback loop infrastructure is fully built and scoring works across five weighted dimensions: completeness, accuracy, flag quality, no-fabrication, and coherence. Meaningful improvement measurement requires multiple runs with corrected few-shot examples injected into the prompt, which the time constraint did not allow. The baseline composite score established in this submission is 3.20 out of 5.00.

Production deployment. The local architecture relies on Tesseract OCR which is not available on standard cloud hosting. Deployment would require either containerisation with Docker or switching to a cloud OCR service such as Google Document AI or AWS Textract for the handwritten pages. The LLM component would switch from local Ollama to the Groq API, which is already supported via a single environment variable change.

KEY DESIGN DECISIONS

Why hybrid Python plus LLM rather than a pure agent loop. A pure LLM agent loop on 71 pages of noisy OCR text produced consistent hallucinations during development — the model substituted plausible clinical values when it could not parse garbled text. Python extraction is deterministic, auditable, and does not depend on the model's willingness to follow instructions under uncertainty. The LLM adds value for language understanding and conflict detection where its capabilities are actually needed.

Why medical APIs rather than LLM knowledge. Drug names and diagnosis codes retrieved from RxNorm and ICD-10 are grounded in authoritative databases. LLM training data for Indian brand drug names is sparse and unreliable. API retrieval is the correct tool for this lookup problem and eliminates a significant hallucination risk at the most safety-critical part of the output.

Why no LangChain or CrewAI. The agent loop, tool dispatcher, and trace logger are approximately 300 lines of Python. Adding a framework would obscure the decision logic and make it harder to audit — which is the opposite of what a clinical safety system requires. Every tool call, every routing decision, and every flag is explicitly coded and visible.

Why SQLite for the feedback loop. Zero infrastructure, zero cost, and built into Python. Sufficient for the scale of this system and trivially upgradeable to PostgreSQL for a production deployment.